Computing Science (CMPUT) 455

Search, Knowledge, and Simulations

Martin Müller

Department of Computing Science
University of Alberta
mmueller@ualberta.ca

Fall 2026

Assignment 2 Preview

- Goals:
  - Write a **perfect solver** for Heap Go
  - Make it respect a given time limit
  - Make it reasonably efficient
  - Use it to `solve` heap Go positions
  - Use solver to improve the `genmove` command of a random player
- Assignment specification: Canvas
- We will talk about the concepts and algorithms in class in the next 4-5 lectures
- I'll try to publish slides early
  - Lecture 7 and 8 slides published now, 9 and 10 "soon"

Perfect Solver for Heap Go

- What does “perfect solver” mean, exactly?
- In this assignment, we only care about **win/loss** (boolean) outcomes
- Here, perfect means:
  - Your solver can find out if a given Heap Go position is a win or a loss...
  - ...with **best play by both** players
  - If a game can be won, then your solver finds a winning move, and your player plays it
  - If a game is lost, your player plays a random legal move
  - If your solver *times out*, your player also plays a random legal move
- We do not care about the exact Heap Go final score here
  - All wins are equally good: a win by 0.5 is as good as a win by some higher score
  - All losses are equally bad

Assignment 2 Coding

- You are given a “clean” Heap Go implementation with the `a2.py` starter code
  - Slightly adapted from an Assignment 1 solution
  - It implements the rules but is not quite ready for search
- Many sample codes in our github repo
  - Boolean minimax and negamax algorithm
  - Examples - solve 21 game, win-no win solver in Tic Tac Toe
  - More code examples “soon”

Assignment 2 Testing and Evaluation

As in Assignment 1:

- For marking/testing: both public and private test cases
- You will benefit from testing your own code very well, including writing your own test cases
- Same type of presubmission test + submit procedure

New in Assignment 2:

- Respect a given time limit for each `solve` call
- There *can* be up to 2 bonus marks for *exceptional* solvers that solve more, *clearly harder* problems than others

Assignment 2 - First Steps

(These are the recommended steps but of course you can do it your own way)

- First step: make a simple solver similar to our sample codes in repo
- Heap Go implementation: from your assignment 1, or use our starter code
- Add extra functions needed in a solver, such as undo move
- Add simple search, such as boolean negamax
- Result: a correct solver that can handle some easy test cases

Assignment 2 - Time Control

- Next, make your solver respect the `timelimit` command
- How? Your choice. Also see the link in the spec - *Measuring Time in Python*
- Test: try some difficult positions with different time limits
- Result: a correct solver that can finish all test cases (some stop with timeout, but they **do** stop in time)
- Implement the player (`genmove` command) - uses the solver and *falls back* to random play if no win found

Assignment 2 More Hints - Optimisations

- Make your solver solve more problems in less time
- How? many possible ways
- Improve raw speed
  - Where does your program spend time? - use profiling
  - Can you find smarter ways, save computation?
  - Pre-computation?
- Better algorithms, game-specific or generic
- Many possible approaches discussed in course slides - Which will work for this game?
- Many more possible approaches - find your own ways
- Test correctness and improvement after each change, make your own test cases
- Results: series of still correct solvers, that solve more and more test cases without timing out

Assignment 2 More Hints - Reduce State Space

- Smaller state space *often* means a faster solver
  - There are *often* trade-offs:
  - smarter = slower (in nodes/second)
  - smarter = more complex, higher chance of bugs creeping in
  - Always keep a backup plan - the best *working* version
- Transposition Table, use DAG structure
  - See lectures
  - Likely very effective here
  - Careful with correctness (e.g. hash collisions)
- Add knowledge: recognise wins/losses early
  - Reduce *effective* search depth
  - Example: one player is leading by more than the value of all remaining tokens
- Use symmetry (can we ???)

Assignment 2 More Hints - Move Ordering

- Move ordering: search (likely) good moves first
- Probably very important here
- Simple example: *greedy* heuristic
  - Sort moves by the value of the tokens taken
- Enjoy solving Heap Go!